CSES — Little Bird

Rating: N/A

1. Statement

There are n trees in a row, and tree i has height d_i. A bird starts at tree 1 and wants to reach tree n. The integer k is the maximum distance the bird can fly in one flight: from tree i, it can fly to any of the trees $i+1, i+2, \ldots, i+k$, rest there, and then start another flight. A flight to a tree that is taller than or as tall as the current one is *tiring*. The goal is to minimize the number of tiring flights needed to reach tree n from tree 1.

There are q queries, each with a different value of k. For each query, determine the minimum number of tiring flights.

2. Solution

A DP approach comes to mind naturally. Let dp_i be the minimum number of tiring flights needed to reach tree i from tree 1. The initial value is $\mathrm{dp}_1 = 0$, and the answer is dp_n. For the transition, tree i can be reached by a flight from any $j \in [i-k, i-1]$, so

$$\mathrm{dp}_i = \min_{j \in [i-k, i-1]} \big(\mathrm{dp}_j + \big[d_j \le d_i\big]\big),$$

where $\big[d_j \le d_i\big]$ is 1 if the bird flies up (or stays at the same height) from j to i, which makes the flight tiring, and 0 otherwise.

However, this transition gives an overall complexity of $O(qnk)$, which is too slow. The sliding window technique can be applied here, with a monotonic queue maintaining the candidates for j.

Since a flight adds at most 1 to the number of tiring flights, the height only serves as a tie-breaker. If $\mathrm{dp}_{j_1} < \mathrm{dp}_{j_2}$, then $\mathrm{dp}_{j_2} - \mathrm{dp}_{j_1} \ge 1$, while the flight from j_1 can cost at most 1 more than the flight from j_2. Therefore, regardless of the heights,

$$\mathrm{dp}_{j_1} + \big[d_{j_1} \le d_i\big] \le \mathrm{dp}_{j_2} + \big[d_{j_2} \le d_i\big].$$

This gives a neat way to order the indices in the queue: if $\mathrm{dp}_{j_1} < \mathrm{dp}_{j_2}$, tree j_1 is considered better (smaller) than tree j_2. If $\mathrm{dp}_{j_1} = \mathrm{dp}_{j_2}$, the tree with the greater height is considered better, since a greater height gives a larger chance of flying down from it.

The best index is at the front of the queue. There is one more restriction: the length of a flight. If the index at the front is more than k away from i, it is invalid for tree i and for every tree after it, so it is popped from the queue. Now the front of the queue (let its index be j) is safe to use for the update:

$$\mathrm{dp}_i = \mathrm{dp}_j + \big[d_j \le d_i\big].$$

To push the new index i into the queue, the trees that are no better than tree i (those with a greater dp value, or an equal dp value and a height not greater than d_i) are removed from the back first, and then i is pushed to the back. This keeps the queue monotonic. Since each tree is pushed once and popped at most once, a query takes $O(n)$ queue operations, and the total time complexity is $O(qn)$.

3. Code

```
#include<bits/stdc++.h>
using namespace std;
int n,d[1000005],Q,k,dp[1000005];
pair<int,int> p;
int main(){
  cin>>n;
  for(int i=1;i<=n;i++) cin>>d[i];
  cin>>Q;
  while(Q--){
    cin>>k;
    dp[1]=0;
    deque<pair<int,int> > q;
    p.first=1,p.second=0;
    q.push_back(p);
    for(int i=2;i<=n;i++){
      while(!q.empty()&&q.front().first<i-k) q.pop_front();
      dp[i]=q.front().second+(d[q.front().first]<=d[i]);
      while(!q.empty()&&q.back().second>dp[i]) q.pop_back();
      while(!q.empty()&&q.back().second==dp[i]&&d[q.back().first]<=d[i]) q.pop_back();
      p.first=i,p.second=dp[i];
      q.push_back(p);
    }
    cout<<dp[n]<<endl;
  }
  return 0;
}
```